**Рішення щодо рефакторингу**

**1. Модулі**

Під час виконання роботи було проведено рефакторинг застосунку Node.js + Express. Початковий код знаходився в одному файлі app.js, тому його було розділено на чотири модулі.

Файл app.js відповідає за налаштування та запуск сервера, routes.js – за обробку HTTP-запитів, service.js – за перевірку даних, обчислення вартості та створення замовлень, а store.js – за зберігання, пошук і видалення замовлень у пам'яті.

Такий поділ дозволив зробити код зрозумілішим і спростити його подальшу підтримку.

**2. Очищення коду**

Під час рефакторингу було видалено непотрібні частини коду. Функція slopify() лише виводила повідомлення в консоль і не впливала на роботу API. Функція unusedLegacyOrderCode() та константа LEGACY_REGION не використовувалися в застосунку.

Також було видалено функцію synchronizeOrderState(), яка створювала штучну затримку від 900 до 2000 мс, та допоміжні функції, пов'язані з нею. Ці затримки не були потрібні для зберігання замовлень у пам'яті.

Після очищення код став простішим, а робота застосунку значно швидшою.

**3. API-контракт**

Для перевірки роботи застосунку використовувалася команда npm.cmd test. До рефакторингу всі 4 тести пройшли успішно, помилок не було. Після внесення змін повторне тестування також показало 4 успішні тести без помилок.

Було збережено HTTP-маршрути та методи, формати запитів і відповідей, коди стану, правила валідації, розрахунок загальної вартості замовлення та зберігання даних у пам'яті.

Таким чином, автоматизовані тести підтвердили збереження перевіреної ними поведінки API.

**4. Швидкодія**

Для перевірки швидкодії застосунку використовувалася команда npm.cmd run load. До рефакторингу було виконано 750 запитів, усі успішні. Середня затримка становила 1190 мс, p50 – 1337 мс, p95 – 1948 мс, p99 – 1997 мс, а пропускна здатність – 20,5 запитів за секунду.

Після рефакторингу також було виконано 750 запитів без помилок. Середня затримка зменшилася до 17 мс, p50 становила 16 мс, p95 – 26 мс, p99 – 36 мс, а пропускна здатність збільшилася до 1455 запитів за секунду.

До змін найповільнішими були ендпоінти GET /orders/:id із середньою затримкою 1491 мс та POST /orders із затримкою 1476 мс. Причиною була штучна затримка у функції synchronizeOrderState(). Після її видалення середня затримка обох ендпоінтів становила 17 мс.

**5. Перевірка AI-змін**

Під час виконання роботи використовувався ChatGPT, який запропонував розділити код на окремі модулі та видалити непотрібні функції.

Однією із запропонованих змін було видалення функції synchronizeOrderState(), яка створювала штучну затримку. Цю зміну було перевірено за допомогою автоматизованих тестів і тесту навантаження.

Після внесення змін усі 4 тести пройшли успішно, а середня затримка запитів значно зменшилася. Тому запропоновану зміну було прийнято.